

React API Integration Guide

A Simplified Explanation of API Integration in React Applications

1. Finding the API Link

- Look at your Scalar or Swagger documentation to find the specific URL for your API.
- You'll find it at the top of the request section.

```
const url = "https://localhost:7132/api/Role";
```

2. Setting Up Your Data Structure

- Important: You need separate states to manage data, screen views, and re-fetching.
- Make sure the property names in your object match exactly with what the backend expects (like roleName and description).

```
// 1. State for Displaying All Records (Array)
const [roles, setRolesState] = useState([]);

// 2. State for Adding/Editing a Single Record (Object)
const [role, setRole] = useState({
  "roleName": "",
  "description": ""
});

// 3. State for Refreshing Data
const [refresh, setRefresh] = useState(true);

// 4. State for Managing Screens
const [viewMode, setViewMode] = useState('list'); // list, add, edit
```

3. Displaying All Records

- Use a `useEffect` hook to fetch the data from the API as soon as the page loads.
- Then, use the `.map()` function to go through the fetched roles array and create table rows (`<tr>`) inside your `<tbody>`.

```
// Fetching Data on Page Load
useEffect(() => {
  fetch(url)
    .then(res => res.json())
    .then(res => setRolesState(res))
}, [refresh]);

// Displaying the List
<tbody>
  {roles.map((role, idx) => (
    <tr key={role.roleId}>
      <td>{idx + 1}</td>
      <td>{role.roleName}</td>
      <td>{role.description}</td>
      <td><!-- Edit/Delete Buttons --></td>
    </tr>
  ))}
</tbody>
```

4. Adding a New Record

- As the user types in the input fields, use the `onChange` event and the spread operator (`...role`) to update your single role state with the new values.
- Finally, send a POST request to the API with the new data to save it.

```
// Update state when typing
<input
  value={role ? role.roleName : ""}
  onChange={(e) => setRole({ ...role, roleName: e.target.value })}
/>

// Save the new record
if (viewMode === "add") {
  fetch(url, {
    method: "POST",
    body: JSON.stringify(role),
    headers: { 'Content-Type': 'application/json' }
  }).then(res => {
    setRefresh(!refresh); // use for get fresh data after changes
    setViewMode('list');  // Returns to the list screen
  })
}
```

5. Editing an Existing Record

- When the user clicks an edit button, fetch the data for that specific record using its ID.
- Put that data into your role state using `setRole(res)`.
- Bind this data to your input fields using the `value` attribute so they automatically fill in.
- After making changes, send a PUT request to save them. Make sure to add the `roleId` to the end of the URL!

```
// 1. Fetch & Pre-fill the state
const handleEdit = (roleId) => {
  fetch(url + "/" + roleId)
    .then(res => res.json())
    .then(res => {
      setRole(res);
      setViewMode("edit");
    })
};

// 2. Bind the data to your input fields

```

6. Deleting a Record

- Send a DELETE request to the API, including the ID of the item you want to remove.
- Always show a confirmation message (like `confirm("Are you sure...")`) before deleting to prevent mistakes.

```
<button onClick={() => {  
  if (confirm("Are you sure you want to delete this role?")) {  
    fetch(url + "/" + role.roleId, {  
      method: "DELETE",  
    }).then(res => {  
      setRefresh(!refresh); // use for get fresh data after changes  
    })  
  }  
}> Delete </button>
```

 **Note on Refreshing Data:** The logic used to fetch fresh data after making changes (like `setRefresh(!refresh)`) might be different depending on how your specific project is set up.